

Regression Models Using Gradient Descent

Assignment 03 — Data Mining and Machine Learning

Student	Muhammad Zohaib Khan
Roll Number	BSDSF24M016
Course	Data Mining and Machine Learning (5th Semester)
Source Notebook	Regression_GradientDescent.ipynb
Date	4 October 2026

1. Introduction

This report documents the implementation of polynomial regression models trained with Gradient Descent (GD) on a single-feature dataset, where the input is X and the target is R . Models of degree 1 through 6 were fitted from scratch using NumPy, without any machine learning library for the optimisation. The weight update equations for each model are derived, the final results are reported, and the GD solutions are compared with the closed-form Normal Equation solution obtained in Assignment 02.

2. Dataset and Experimental Setup

Parameter	Value
Training samples (m)	283
Test samples	32
Learning rate (α)	0.01
Iterations	1000
Initial weights	$\theta = 0$ (zero vector)
Feature scaling	None for degrees 1–2; z-score normalisation for degrees 3–6

For degrees 3 to 6, each power of x is standardised using the mean and standard deviation of the *training* set, and the same training statistics are applied to the test set. This prevents information from the test set leaking into training. Normalisation was introduced because the raw powers of x differ greatly in scale, which slows Gradient Descent considerably.

3. Cost Function and Weight Update Equations

3.1 General form

For a model with hypothesis $h_{\theta}(x) = X\theta$, the cost function is:

$$J(\theta) = (1 / 2m) \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

Taking the partial derivative with respect to each weight gives:

$$\partial J / \partial \theta_j = (1 / m) \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)}$$

In vectorised matrix form, the update applied at every iteration is:

$$\theta \leftarrow \theta - \alpha \cdot (1 / m) \cdot X^T (X\theta - y)$$

where X is the $m \times (n+1)$ design matrix whose first column is all ones, and x_j denotes the j -th feature column (the j -th power of x , normalised where indicated).

3.2 Weight update equations for each model

Model 1 (Linear). $h(x) = \theta_0 + \theta_1 x$

$$\theta_0 \leftarrow \theta_0 - \alpha (1/m) \sum (h(x^{(i)}) - y^{(i)})$$

$$\theta_1 \leftarrow \theta_1 - \alpha (1/m) \sum (h(x^{(i)}) - y^{(i)}) x^{(i)}$$

Model 2 (Quadratic). $h(x) = \theta_0 + \theta_1 x + \theta_2 x^2$

$$\theta_0 \leftarrow \theta_0 - \alpha (1/m) \sum (h - y)$$

$$\theta_1 \leftarrow \theta_1 - \alpha (1/m) \sum (h - y) x$$

$$\theta_2 \leftarrow \theta_2 - \alpha (1/m) \sum (h - y) x^2$$

Model 3 (Cubic). $h(x) = \theta_0 + \theta_1 z_1 + \theta_2 z_2 + \theta_3 z_3$, where $z_k = (x^k - \mu_k) / \sigma_k$

$$\theta_j \leftarrow \theta_j - \alpha (1/m) \sum (h - y) z_j^{(i)} \quad \text{for } j = 0, 1, 2, 3 \text{ (with } z_0 = 1)$$

Models 4, 5 and 6. The same pattern extends to any degree n . For degree n with normalised features:

$$h(x) = \sum_{j=0}^n \theta_j z_j, \quad \theta_j \leftarrow \theta_j - \alpha (1/m) \sum_{i=1}^m (h(x^{(i)}) - y^{(i)}) z_j^{(i)}$$

with $z_0 = 1$. Degrees 4, 5 and 6 therefore have 5, 6 and 7 weights, all updated simultaneously in each iteration.

4. Results

4.1 Final learned weights

Model	θ_0	θ_1	θ_2	θ_3	θ_4	θ_5	θ_6
1 (Linear)	0.7512	-0.4903	—	—	—	—	—
2 (Quadratic)	0.5817	-0.1750	-0.0974	—	—	—	—
3 (Cubic)	0.0049	-0.5215	-0.2257	0.2839	—	—	—
4	0.0049	-0.5496	-0.3794	0.0417	0.4505	—	—
5	0.0049	-0.5134	-0.4603	-0.1325	0.2056	0.4892	—
6	0.0049	-0.4538	-0.4844	-0.2321	0.0440	0.2777	0.4603

Table 1. Weights after 1000 iterations of Gradient Descent ($\alpha = 0.01$).

Models 1 and 2 use raw features, so their weights are on the original scale. Models 3 to 6 use normalised features, so θ_0 equals the mean of the training targets (≈ 0.0049) and the other weights are not directly comparable with those of Models 1 and 2.

4.2 Mean Square Error and convergence

MSE is reported using the same $(1/2m)$ convention as the cost function in Section 3, following the course specification. The cost values at each checkpoint use this same convention and show the convergence behaviour.

Model	Train MSE	Test MSE	Cost at iter. 100	Cost at iter. 500	Cost at iter. 1000
1 (Linear)	0.1514	0.1571	0.2211	0.1638	0.1514
2 (Quadratic)	0.1619	0.1610	0.1958	0.1696	0.1619
3 (Cubic)	0.1365	0.1485	0.1654	0.1475	0.1365
4	0.1151	0.1332	0.1674	0.1297	0.1151
5	0.0980	0.1192	0.1638	0.1110	0.0980
6	0.0871	0.1086	0.1566	0.0968	0.0871

Table 2. Gradient Descent results (MSE with $1/2m$ convention).

5. Comparison: Gradient Descent vs. Normal Equation

Assignment 02 solved the same regression problems in closed form using the Normal Equation, $\theta = (X^T X)^{-1} X^T y$. Table 3 compares the resulting MSE values with those from Table 2, both using the $(1/2m)$ convention. The Assignment 02 values were originally computed as $(1/m)$ and have been halved here for a like-for-like comparison.

Model	GD Train	NE Train	GD Test	NE Test	Test MSE gap (GD – NE)
1 (Linear)	0.1514	0.1496	0.1571	0.1580	−0.0009
2 (Quadratic)	0.1619	0.1442	0.1610	0.1630	−0.0020
3 (Cubic)	0.1365	0.0249	0.1485	0.0258	+0.1227
4	0.1151	0.0222	0.1332	0.0250	+0.1082
5	0.0980	0.0193	0.1192	0.0221	+0.0971
6	0.0871	0.0193	0.1086	0.0223	+0.0863

Table 3. MSE comparison of the two approaches (1/2m convention).

For Models 1 and 2, the two approaches agree closely, with differences of about 0.002 or less in MSE. Both have effectively converged to the least-squares solution for their respective feature sets. For Models 3 to 6, however, the Gradient Descent results are far worse than the Normal Equation results, with test MSE roughly five to six times higher. The Normal Equation finds the exact minimum of the cost function in a single step, whereas Gradient Descent only approaches it over successive updates. The cost curves in Table 2 are still decreasing at iteration 1000, which means these models stopped well before reaching the minimum. The gap is therefore a consequence of the chosen learning rate and iteration budget, not evidence that the higher-degree models are unsuitable.

6. Convergence Analysis

- **Models 1 and 2** converge quickly. Their cost falls by about 0.06 in the first 500 iterations and then changes very little, so they are close to their minimum by iteration 1000.
- **Models 3 to 6** converge more slowly. Their cost is still falling steadily at iteration 1000 (for example, Model 6 falls from 0.0968 to 0.0871 between iterations 500 and 1000). A larger learning rate, more iterations, or a convergence-based stopping criterion would be needed to reach the minimum.
- Feature normalisation was essential for Models 3 to 6. Without it, the very large values of x^5 and x^6 would make a single learning rate unsuitable for all weights at once.

7. Overfitting and Underfitting

In this experiment, the Gradient Descent models show **underfitting** rather than overfitting. Train and test MSE both decrease steadily as the polynomial degree increases, and the test MSE is never substantially higher than the training MSE. Because every model stopped before convergence, the extra flexibility of higher-degree models is only partly used. The overfitting trend seen in Assignment 02, where test error rose at degree 6, cannot appear here because the models never fit closely enough to the training noise. Reaching that regime would require training the higher-degree models to convergence, after which the training error would fall further and test error could start to rise.

8. Conclusion

The Gradient Descent implementation reproduces the Normal Equation solution for linear and quadratic regression within a small margin, confirming that the update equations in Section 3 are correct. For cubic through sixth-degree polynomials, Gradient Descent with $\alpha = 0.01$ and 1000 iterations has not converged, and its MSE values are substantially higher than the closed-form values. The Normal Equation is exact and efficient for a problem of this size with few features. Gradient Descent remains valuable for larger datasets and many features, where inverting $X^T X$ becomes expensive, but it requires careful choice of learning rate, feature scaling, and iteration count to match the exact solution.